

BSC (HONS) IN INDUSTRIAL INFORMATION TECHNOLOGY — FINAL YEAR
RESEARCH PROJECT

AI-Based Tomato Ripening Stage Detection and Sorting System with Shelf-Life Estimation for the Commonly Consumed Padma Variety in Sri Lanka Using Computer Vision and Conveyor Automation

Chapter 3: Methodology

W.K. Gayan — UWU/IIT/21/062

K.R.D.H. Gunawardhana — UWU/IIT/21/017

P.L.H. Hirushan — UWU/IIT/21/019

Principal Supervisor: Ms. D.P. Jayathunga

Co-Supervisors: Dr. U.G.A.T. Premathilake, Ms. M.P.A.M. Rathnakumara

Department of Computer Science and Informatics

Faculty of Applied Sciences

Uva Wellassa University of Sri Lanka

Table of Contents

3.1 Introduction

3.2 Overall Research Methodology

3.3 Research Materials

3.3.1 Tomato Samples

3.3.2 Hardware Components

3.3.3 Software and Technologies

3.4 Tomato Sample Collection

3.5 Data Collection and Image Acquisition

3.6 Dataset Description

3.7 Tomato Ripeness Classification

3.8 Image Preprocessing

3.9 Image Annotation

3.9.1 Annotation Tool and Format

3.9.2 Annotation Convention

3.9.3 Annotation Quality Control and Correction

3.10 Dataset Splitting

3.11 AI Model Development

3.12 Model Training

3.12.1 Training Environment

3.12.2 Model Configuration and Hyperparameters

3.12.3 Training Procedure

3.13 Model Validation

3.14 Model Testing and Evaluation

3.14.1 Evaluation Metrics

3.14.2 Test-Set Results

3.15 Tomato Detection Process

3.16 Hardware Prototype Development

3.16.1 Mechanical Structure and Conveyor Mechanism

3.16.2 Camera Mounting

3.16.3 Sensors and Sorting Mechanism

3.16.4 Controller and Power

3.16.5 Completion Status

3.17 Software System Development

3.18 Hardware–Software Integration

3.19 Complete End-to-End System Process

3.20 Experimental Procedure

3.21 System Testing

3.22 Data Analysis

3.23 Research Workflow Diagram

3.24 Ethical and Safety Considerations

3.25 Limitations of the Methodology

3.26 Summary of Figures and Tables

3.27 Appendix Guidance

Note on this draft. This chapter was assembled directly from your project repository (`D:\Research\Tomato_Projet_five_class--Deployement-Model--Tomato-Research--`), your seven monthly progress reports (Jan–Jul 2026), `PROJECT.md` , the training logs for the currently deployed model (`runs_local/tomato_5class_v6_balanced/`), and — as of this revision — the full text of `Tomato_Progress.pdf` (the formal progress report submitted to your supervisors, extracted and read in full). Every number in this draft is traceable to one of those sources — none are estimated or invented. Anywhere the source material did not contain a fact needed for the chapter, it is marked **[INFORMATION REQUIRED]** with a short note on what is needed.

`Tomato_Progress.pdf` resolved several previously-open items — most notably the exact tomato-collection site (§3.4) and the full shelf-life observation protocol (§3.20, Experiment 4), which turns out to derive exactly the `database.py` shelf-life values (31/29/24/12 days). It also revealed that this PDF's §4 ("Methodology") and its dataset-size target (>9,600 images) and split ratio (70/10/30) describe the **original proposed plan**, which differs from what was actually executed (7,849 images; ~67/19/9.5% split) — both are now reported side-by-side where relevant, since reporting only the achieved figures without noting the planned target would understate a real, citable deviation.

One thing remains unresolved and should be checked before submission: - **A wiring conflict exists between two files** in the `tomato_v2` firmware regarding physical gate order (see §3.16.3) — flagged there, needs a quick check against the physical rig.

Delete this note before submitting the chapter.

Chapter 3: Methodology

3.1 Introduction

This chapter describes the research approach and complete system-development process followed in building an AI-based ripening-stage detection and sorting system for the Padma tomato variety commonly consumed in Sri Lanka. The study combines an applied computer-vision development methodology (dataset construction, model training, and empirical evaluation) with an engineering-prototyping methodology (mechanical, electronic, and firmware development), integrated into a single conveyor-based hardware prototype.

The chapter is organised to follow the actual development sequence of the project: tomato sample collection and image acquisition; dataset preparation, annotation, and splitting; deep-learning model development, training, validation, and testing; hardware prototype design and fabrication; software system development; hardware–software integration; and finally the experimental and testing procedures used to evaluate the complete system. Each stage is described in terms of what was done, why it was done, how it was implemented, and what evidence (data, figures, logs) supports it, so that the work is reproducible by another researcher.

Three registered students contribute to this project under a shared title and shared supervision (Ms. D.P. Jayathunga, Principal Supervisor; Dr. U.G.A.T. Premathilake and Ms. M.P.A.M. Rathnakumara, Co-Supervisors), each responsible for a distinct sub-system: W.K. Gayan (UWU/IIT/21/062) — deep-learning model development and real-time inference/hardware integration; K.R.D.H. Gunawardhana (UWU/IIT/21/017) — shelf-life prediction and KPI monitoring; P.L.H. Hirushan (UWU/IIT/21/019) — mechanical conveyor prototype and actuation. This chapter is written from the perspective of, and documents in most detail, the model-development and integration work (UWU/IIT/21/062); the shelf-life estimation and mechanical-design sub-systems are described only to the extent that their outputs and interfaces affect the integrated system, consistent with the division of work across the team.

3.2 Overall Research Methodology

The research followed an applied, iterative development methodology rather than a single-pass linear pipeline: the dataset, annotation conventions, and model were each revised at least once in response to evidence gathered during testing (see §3.9.3 and the defect-class case study in §3.13.3). The overall stages, in the order they were executed, were:

Problem definition → Tomato sample collection → Image acquisition → Dataset preparation (Roboflow) → Image annotation (Roboflow) → Dataset splitting (train/valid/test) → Image preprocessing and augmentation → AI model development (YOLOv8) → Model training (local GPU) → Model validation → Model testing (held-out test set) → Performance evaluation → Iterative dataset/model correction (where testing revealed a defect) → Hardware prototype design and fabrication → Software system development → Hardware–software integration → Real-time tomato detection → Ripeness-stage classification → Sorting decision and actuation → System-level evaluation.

INSERT FIGURE 3.1: OVERALL RESEARCH METHODOLOGY FLOWCHART

Figure 3.1. Overall research methodology, from problem definition to system evaluation.

Recommended flowchart structure for Figure 3.1 (to be converted into a diagram):

```

Problem Definition
|
Tomato Sample Collection (collecting center, Jan 2026 onward)
|
Image Acquisition (LED lightbox, white background)
|
Dataset Preparation (Roboflow)
|
Image Annotation (Roboflow, whole-tomato bounding boxes, 5 classes)
|
Dataset Splitting (train / valid / test)
|
Image Preprocessing & Augmentation
|
AI Model Development (YOLOv8m, object detection)
|
Model Training (local RTX 3070 Ti)
|
Model Validation (validation split, early stopping)
|
Model Testing (held-out test split)
|
Performance Evaluation (precision, recall, mAP50, mAP50-95, confusion matrix)
|
+--> [if evaluation reveals a systematic weakness] --> Dataset/Annotation Correction --> re-Training
(see §3.13.3)
|
Hardware Prototype Development (conveyor, stepper drive, sorting gates, sensors)
|
Software System Development (Streamlit dashboard, SQLite logging, serial communication)
|
Hardware-Software Integration (Bluetooth/serial link, ESP32 firmware)
|
Real-Time Tomato Detection (camera -> YOLOv8 inference)
|
Ripeness-Stage / Defect Classification (per-tomato track voting)
|
Sorting Decision (class -> gate command)
|
Mechanical Sorting (servo gate actuation)
|
System-Level Evaluation (functional, integration, and end-to-end testing)

```

3.3 Research Materials

This section describes every material — biological sample, hardware component, and software tool — used in the research, consistent with the requirement that all materials and samples be explained and supported with figures and tables.

3.3.1 Tomato Samples

Padma-variety tomatoes were the biological material of the study. Images were captured at the **CLD Farm Fresh collecting center, Bandarawela** (Badulla District, Uva Province, Sri Lanka), where Padma tomatoes arrive from multiple cultivation areas across the Badulla region — chosen specifically because sourcing from a multi-farm collecting center captures natural variation in fruit size, surface texture, maturity progression, and handling-related defects while still allowing a single, consistent capture setup (Source: `Tomato_Progress.pdf`, §4.2 "Collection Site and Sampling"; January 2026 Progress Report, UWU/IIT/21/062). Photography itself was performed using a white-box lightbox with LED illumination to keep image quality consistent across sessions. Tomatoes were visually sorted at the time of capture into the project's five target categories (four ripeness stages plus a defect category, defined in §3.7) so that Roboflow annotation could proceed directly from the sorted image sets.

[INFORMATION REQUIRED] — the following sample-level details are still not recorded in the repository, the progress reports, or `Tomato_Progress.pdf`, and must be supplied before this subsection can be finalised: - Total number of physical tomato samples/units photographed (as distinct from the final image count, which includes multiple images per tomato from different angles/sessions — see §3.6). - Exact collection dates/sessions (progress reports confirm collection began in January 2026 and that "additional images... in varied lighting conditions" were planned as of the April 2026 reporting period, but do not give exact session dates). - Precise inclusion/exclusion criteria used when selecting tomatoes for each class (e.g. minimum/maximum size, specific defect types accepted under "defect"). - Sample storage and transportation conditions between collection and photography, if any interval existed.

Table 3.1. Materials and samples used in the research.

Material/Sample	Description	Source	Quantity	Purpose
Padma tomato fruit	Fresh tomato units at varying ripeness stages (green, breaker, turning, red) and with visible defects (cracks, bruising, rot)	CLD Farm Fresh collecting center, Bandarawela, Badulla District	[INFORMATION REQUIRED: total physical sample count]	Source material for image dataset (§3.5) and for physical conveyor testing (§3.20–3.21)
Publicly sourced defect-class images	Third-party tomato defect images incorporated into an earlier dataset version, later identified as a source of a domain/lighting mismatch and removed/supplemented (see §3.9.3, §3.13.3)	Kaggle (public dataset)	Not separately recorded; superseded by self-captured images in the current (v6) export	Historical — no longer part of the deployed model's training data

3.3.2 Hardware Components

Table 3.2. Hardware components used in the physical prototype, consolidated from the project's procurement records (July 2026 Progress Report) and the component list you supplied. Quantities are given where confirmed; unconfirmed quantities are marked accordingly.

Component	Model/Specification	Quantity	Purpose
Microcontroller	ESP32 development board (DOIT ESP32 DEVKIT V1, per <code>tomato_v2/platformio.ini</code> board target <code>esp32doit-devkit-v1</code>)	1	Main sorting controller — runs <code>tomato_v2</code> firmware; receives classification commands over Bluetooth SPP and drives the stepper motor, 4 gate servos, and 4 IR sensors
Conveyor drive motor	NEMA 23 stepper motor	1	Drives the conveyor belt
Stepper motor driver	DM542 / DM543 [INFORMATION REQUIRED: confirm exact model — July progress report states "DM543", earlier project documentation and code comments state "DM542"]	1	Converts step/direction signals from the ESP32 into motor drive current for the NEMA 23
Belt-side power supply	24 V, 5 A PSU	1	Powers the stepper driver
Logic-side power adapter	12 V, 2 A adapter	1	Auxiliary power
Sorting-gate actuators	MG995 servo motor	4	One gate per class (green/breaker/turning/red); opens to divert a tomato off the belt at the correct point (defect receives no gate command and continues to the belt end, per design)
Object-presence sensors	IR (infrared) sensor	4	Detects tomato arrival at each gate position for sequencing (<code>tomato_v2</code> firmware, GPIO 18/19/21/23)
Power-transmission — pulley (motor side)	8 mm bore, 20-tooth pulley	1	Mounted on stepper motor shaft
Power-transmission — pulley (roller side)	10 mm bore, 60-tooth pulley	1	Mounted on conveyor drive roller
Power-transmission — belt	Closed-loop timing belt	1	Connects motor and roller pulleys
Motor mount	L-shaped bracket	1	Secures NEMA 23 to the frame
Servo mounts	3D-printed servo mount bracket	4	Secures each MG995 gate servo
IR sensor mounts	3D-printed IR sensor mount	4	Secures each IR sensor at its gate position
Sorting blade	3D-printed sorting blade	1	Physical diverter actuated by each gate servo
Conveyor frame	Metal box-section frame (1-inch box bar), painted finish	1	Structural support for the conveyor and all mounted components

Component	Model/Specification	Quantity	Purpose
Conveyor rollers/bearings/brackets	Custom-fabricated bracket assemblies with integrated bearings	6 (brackets)	Support smooth roller rotation
Camera mount	Adjustable single-axis camera mounting mechanism (custom-fabricated)	1	Allows camera position/angle adjustment for field-of-view optimisation
Camera	USB webcam [INFORMATION REQUIRED: exact model — described only generically as a "high-resolution camera (e.g. USB HD camera)" in Tomato_Progress.pdf §2 and §4.6; code opens it generically via cv2.VideoCapture(0) at 960×720 (@ 30 FPS)]	1	Captures the live video feed used for detection
Support wheels	"3-wheel" castors [INFORMATION REQUIRED: confirm intended use — listed in your component notes without further detail]	3	[INFORMATION REQUIRED]
Perfboard / prototyping board	Dot board	[INFORMATION REQUIRED]	General circuit prototyping
Header pins	40-pin female header, 41-pin male header	[INFORMATION REQUIRED]	Connector interfacing
DC power connectors	DC socket PCB, DC jack	1 each [INFORMATION REQUIRED: confirm]	Power input connection
Switch	On/off switch	[INFORMATION REQUIRED]	Power control
Heat sink	[INFORMATION REQUIRED: spec]	[INFORMATION REQUIRED]	Thermal management (likely for the stepper driver or voltage regulator)
Indicator/lighting components	Bulb	3	[INFORMATION REQUIRED: exact function — status indication or lightbox lighting]
Diode	[INFORMATION REQUIRED: type]	3	Circuit protection
Resistor	1 kΩ	5	Circuit biasing/current-limiting
Resistor	2 kΩ	6	Circuit biasing/current-limiting
Potentiometer	[INFORMATION REQUIRED: value]	1	Manual speed/parameter control (wired to ESP32 GPIO 34 in tomato_v2 firmware)
Voltage regulator	Step-down ("buck") regulator	[INFORMATION REQUIRED]	Voltage regulation for logic-side components
Cabling/connectors	Circuit wire, jumper wires, cable protectors, copper pins, micro-USB cable, solder wire	As needed	Wiring and inter-module connections

Component	Model/Specification	Quantity	Purpose
Fasteners/adhesives	Nails, nuts, bolts, super glue, rubber bush	As needed	Mechanical assembly
Computer (training + inference host)	PC with NVIDIA GeForce RTX 3070 Ti (8 GB VRAM)	1	Local model training (§3.12) and, when used as the inference host, real-time detection

Two items from your original list — "Eliyata yana thahadu tikata" (transport tickets) and "transport (Bus/Luggage)" — are logistics costs, not physical system components, and are therefore omitted from Table 3.2; they belong in a project budget/cost appendix rather than the materials table.

INSERT FIGURE 3.2: HARDWARE COMPONENTS AND PROTOTYPE ASSEMBLY

Figure 3.2. Key hardware components of the conveyor sorting prototype. Source images available in `prototype design/` (CAD renders from Onshape and WhatsApp photographs of the assembled frame, motor, and belt drive, 2026-07-23) and the July 2026 progress report (motor/pulley/belt assembly photographs).

3.3.3 Software and Technologies

Table 3.3. Software, frameworks, and libraries used, taken directly from `requirements.txt` (header comment: "confirmed working together on this project as of 2026-07-23, Python 3.11.9, RTX 3070 Ti").

Software/Technology	Version	Purpose
Python	3.11.9	Primary programming language for model training, inference, dashboard, and hardware-bridge code
PyTorch	2.11.0+cu128	Deep-learning framework underlying YOLOv8
torchvision	0.26.0+cu128	Vision-model utilities used by the training/inference stack
Ultralytics (YOLOv8)	8.4.104	Object-detection model architecture, training, validation, and inference API
OpenCV (opencv-python)	5.0.0.93	Camera capture, frame processing, bounding-box drawing
Streamlit	1.60.0	Web-based dashboard UI (<code>dashboard_app.py</code>)
Plotly	6.9.0	Interactive charts in the dashboard (class-distribution pie chart)
pandas	3.0.5	Tabular data handling (detection logs, dataset audit)
NumPy	2.4.4	Numerical operations
PyYAML	6.0.3	Reading dataset/config YAML files (<code>data.yaml</code> , <code>data_balanced.yaml</code>)
pyserial	3.5	Serial/Bluetooth-SPP communication with the ESP32 (imported lazily; the pipeline runs without hardware attached if unavailable)

Software/Technology	Version	Purpose
SQLite (via Python's built-in <code>sqlite3</code>)	Bundled with Python 3.11.9	Local detection/KPI database (<code>tomato_detections.db</code>)
Roboflow	Web platform (SaaS, version not applicable)	Dataset annotation, class management, and versioned export (workspace <code>tomatoproject-sshvc</code> , project <code>tomato_project-betna</code>)
PlatformIO	[INFORMATION REQUIRED: exact CLI/Core version]	Firmware build/upload toolchain for the ESP32 (<code>tomato_v2</code> project)
Arduino framework (via PlatformIO)	[INFORMATION REQUIRED: framework version pinned by PlatformIO]	ESP32 firmware base framework
ESP32Servo (Arduino library)	^3.0.5	Servo control from ESP32 firmware
CUDA Toolkit	12.8 (cu128 build of PyTorch/torchvision)	GPU acceleration for training and inference
SolidWorks	[INFORMATION REQUIRED: version]	3D CAD design of the conveyor prototype (used Jan–Apr 2026 per progress reports)
Onshape	Web CAD platform	Exported prototype CAD models (<code>prototype design/prototype desing</code> exported from <code>cad.onshape.com/</code>)

3.4 Tomato Sample Collection

Tomato samples were collected at the **CLD Farm Fresh collecting center, Bandarawela** (Badulla District), a multi-farm collection point where Padma tomatoes arrive from cultivation areas across the Badulla region, using a **purposive sampling strategy**: rather than sampling tomatoes at random, images were deliberately captured across all five target categories (green, breaker, turning, red, defect) to ensure the resulting dataset had adequate representation of every class the detection model needed to learn, including the comparatively rare defect condition. Sourcing from a multi-farm collecting center, rather than a single farm, was itself a deliberate choice to capture natural variation in fruit size, surface texture, and maturity progression within each class (`Tomato_Progress.pdf` , §4.2). Purposive sampling — rather than random or stratified probability sampling of the general tomato population — is the methodologically appropriate approach here because the research objective is model generalisation across defined visual categories for a supervised object-detection dataset, not population-level statistical inference about Sri Lankan tomato ripeness distributions at large.

Collection began in January 2026, using a white-box lightbox with LED lighting to standardise background and illumination across capture sessions (January 2026 Progress Report). A further round of image collection, specifically targeting "varied lighting conditions" to improve the model's robustness, was planned for the April–May 2026 period (April 2026 Progress Report) — this directly relates to the domain/lighting mismatch problem discovered and corrected later in the project (§3.9.3).

[INFORMATION REQUIRED]: exact total sample count, precise session dates, and formal inclusion/exclusion criteria per class, as listed in §3.3.1 (the collection site/address is now confirmed above).

INSERT PHOTOGRAPH OF TOMATO SAMPLE COLLECTION PROCESS

Figure 3.3. Tomato sample collection process at the collecting center, using a white-box LED lightbox for consistent image capture.

3.5 Data Collection and Image Acquisition

Images were acquired using a lightbox setup: tomatoes were placed against a white background under LED lighting to minimise shadow and colour-cast variation between capture sessions (January 2026 Progress Report). This controlled setup was used for four of the five classes (green, breaker, turning, red); the defect class initially drew additional images from a public Kaggle dataset with varied, uncontrolled backgrounds and lighting, which was later found to bias the model and was corrected (§3.9.3, §3.13.3) — the corrected/current defect data was re-captured under the same self-controlled conditions as the other four classes.

For the real-time deployment/inference path (as opposed to the static training-image capture above), the system uses a USB webcam connected to the host PC, opened via OpenCV at a resolution of 960×720 pixels and 30 FPS (`conveyor_integration.py` , `dashboard_app.py`), with camera auto-exposure/auto-white-balance allowed to settle for approximately 1.5 seconds after initialisation before frames are used for inference.

[INFORMATION REQUIRED]: - Exact camera/device model used for dataset image capture (may differ from the live-inference webcam). - Image resolution and file format at capture time (the dataset's stored images should be checked directly — e.g. via file properties — for exact pixel dimensions and format, likely JPEG). - Camera distance from the tomato and camera angle/orientation during dataset capture. - Whether multiple orientations of each tomato (e.g. rotated/flipped views of the same fruit) were deliberately captured, and how many. - Any image-quality control step performed at capture time (e.g. discarding blurred/out-of-focus images) beyond the lightbox's controlled lighting.

INSERT FIGURE 3.4: IMAGE ACQUISITION SETUP — LIGHTBOX, LIGHTING, AND CAMERA ARRANGEMENT

Figure 3.4. Image acquisition setup used for dataset capture (LED lightbox, white background) and the live-inference camera arrangement mounted on the conveyor (adjustable single-axis mount, custom-fabricated May–June 2026).

**INSERT FIGURE 3.5: EXAMPLE CAPTURED TOMATO IMAGES ACROSS RIPENESS STAGES
AND THE DEFECT CLASS**

Figure 3.5. Representative sample images for each of the five target classes.

3.6 Dataset Description

The dataset used to train the currently deployed model is the Roboflow export `tomato_project.v6-version_05.yolov8` (workspace `tomatoproject-sshvc`, project `tomato_project-betna`, Roboflow version 6, licensed CC BY 4.0), defined by `data.yaml` and, for the balanced training configuration actually used, `data_balanced.yaml`.

Table 3.4. Dataset summary.

Property	Value
Number of classes	5
Class names	breaker, defect, green, red, turning
Training images (base export)	5,498
Training images, defect-only augmentation supplement	668 (offline-augmented, train split only — see §3.8)
Validation images	1,574
Test images (held out)	777
Total distinct captured/exported images (train + valid + test)	7,849
Image format	[INFORMATION REQUIRED: confirm exact format, e.g. JPEG]
Image resolution as stored	[INFORMATION REQUIRED: confirm — model is trained at 640×640 after resizing, but native captured resolution should be reported]
Annotation format	YOLO (normalised bounding box: class, x-center, y-center, width, height)
Dataset host/versioning	Roboflow

Table 3.5. Per-class object-instance counts by split (one tomato may appear as one annotated instance per image; counted directly from the YOLO label files).

Class	Train (base)	Train (augmented defect supplement)	Validation	Test	Total instances
breaker	1,137	0	326	191	1,654
defect	448	896	137	66	1,547
green	1,380	0	429	191	2,000
red	1,434	0	395	190	2,019
turning	1,138	0	309	149	1,596
Total	5,537	896	1,596	787	8,816

Note that instance counts differ slightly from image counts because a single image can contain more than one annotated tomato.

INSERT FIGURE 3.6: DISTRIBUTION OF IMAGES/INSTANCES ACROSS RIPENESS CLASSES

Figure 3.6. Number of annotated instances per class, by dataset split (bar chart recommended, using Table 3.5).

INSERT FIGURE 3.7: TRAINING/VALIDATION/TESTING DISTRIBUTION

Figure 3.7. Proportion of the dataset allocated to training, validation, and testing (pie or stacked bar chart, using Table 3.6 in §3.10).

A comparable, earlier Roboflow export (`tomato_project.v5-version_04.yolov8`: 5,927 train / 1,692 valid / 845 test images) exists in the repository but is **not** the dataset used to train the currently deployed model — it predates the defect-class domain-mismatch correction described in §3.9.3 and is retained only for internal comparison, not as part of the reported methodology.

Deviation from the originally proposed dataset target. The project's formal research proposal (`Tomato_Progress.pdf`, §4.2 "Dataset Size Target") set a target of "more than 9,600 images" across the five classes. The dataset actually exported and used to train the deployed model (7,849 images total) is smaller than this original target. This is reported here as a factual deviation between the planned and achieved dataset size, not glossed over — if the thesis's own stated objectives (§1.3 of the proposal document, "more than 5,500 Padma tomato images") are being carried into this thesis's objectives section, note that 7,849 images does exceed the lower 5,500-image objective figure, even though it falls short of the later, larger 9,600-image target stated in §4.2 of the same document.

3.7 Tomato Ripeness Classification

The project defines five mutually exclusive target classes: four ripeness stages plus one quality/defect category. The ripeness-stage boundaries are defined by the proportion of red surface colouration, consistent with standard visual tomato-maturity grading conventions.

Table 3.6. Ripeness/quality classes used in this research.

Class	Description	Identification Characteristics
Green	Unripe	Entirely green skin; no red/orange colouration
Breaker	Onset of ripening	Green-to-yellow transition; less than 30% of the surface showing red/pink colouration
Turning	Mid-ripening	30–80% of the surface showing red colouration; mixed green/red/orange patches
Red	Fully ripe	More than 80% of the surface red; uniform ripe colouration

Class	Description	Identification Characteristics
Defect	Rejected quality	Visible cracks, bruising, or rot; may occur at any ripeness stage but is treated as its own class and does not receive a ripeness label

Source: confirmed identical in both `PROJECT.md` (internal project documentation) and `Tomato_Progress.pdf` (Table 1, "Proposed tomato grading classes for the Padma variety"), the formal, authoritative academic document — the class boundaries (Breaker <30% red, Turning 30–80% red, Red >80% red) match exactly across both sources. `Tomato_Progress.pdf` additionally labels these Stage 1–Stage 4 (Green=Stage 1, Breaker=Stage 2, Turning-Red=Stage 3, Red=Stage 4); this thesis follows the descriptive class names (green, breaker, turning, red) used by the Roboflow dataset and the deployed model, which are the same classes under different labels.

INSERT FIGURE 3.8: VISUAL EXAMPLES OF EACH RIPENESS STAGE AND THE DEFECT CLASS

Figure 3.8. Representative images of Green, Breaker, Turning, Red, and Defect tomatoes from the dataset.

3.8 Image Preprocessing

Resizing and normalisation. All images are resized to 640×640 pixels at both training and inference time (`imgsz: 640` in `runs_local/tomato_5class_v6_balanced/args.yaml`; `INFERENCE_IMGsz = 640` in `conveyor_core.py`). This was done because YOLOv8 requires a fixed input size for batched GPU processing, and 640×640 is the architecture's standard operating resolution, balancing detection accuracy against inference speed for real-time use on the conveyor. Pixel-value normalisation (scaling to the [0, 1] range expected by the network) is handled internally by the Ultralytics YOLOv8 pipeline and was not separately implemented.

Data augmentation (training-time). Augmentation was applied on-the-fly during training by Ultralytics' built-in augmentation pipeline, configured with the following parameters (`args.yaml`): HSV colour-space jitter (`hsv_h=0.015`, `hsv_s=0.7`, `hsv_v=0.4`), geometric transforms (`degrees=15`, `translate=0.1`, `scale=0.5`), horizontal flip (`flipplr=0.5`; vertical flip disabled, `flipud=0.0`), mosaic augmentation (`mosaic=1.0`, disabled for the final 10 epochs via `close_mosaic=10`), mixup (`mixup=0.15`), random erasing (`erasing=0.4`), and RandAugment-based auto-augmentation (`auto_augment=randaugment`). This was done to improve the model's robustness to real-world variation in tomato orientation, lighting, and partial occlusion, which is important given that live-camera conditions differ from the controlled lightbox conditions used for capture.

Offline, class-targeted augmentation for the defect class. Because the corrected/re-annotated defect class had substantially fewer training instances (448) than the other four classes (1,137–1,434), an additional, separate offline augmentation step was applied specifically to defect-class training images using the `albumentations` library (script: `augment_defect_class.py`): bounding-box-aware flip,

rotation, scale, brightness, and hue-jitter transforms were applied to the 334 defect-only training images, producing 2 augmented copies per image (668 new images, 896 new annotated instances), bringing the defect class's training-instance count to a level comparable with the other four classes. This was done to correct a class-imbalance problem identified during the domain-mismatch investigation (§3.9.3), without collecting additional real photographs. The augmented images were added only to the training split, referenced via a second `train:` path entry in `data_balanced.yaml`; the validation and test splits were left completely untouched, so that the reported validation/test metrics in §3.14 are not inflated by synthetic data.

Dataset audit (pre-training quality control). Before each training run, `train_local.py`'s `audit_dataset()` function performs an automated preprocessing/quality check: it verifies the class-name order matches the expected 5-class order, confirms a test split is present, computes per-split per-class instance counts, flags orphaned images (images without a corresponding label file) and empty label files, checks for exact-duplicate images leaking across splits (via MD5 hashing), and — specifically added after the annotation-convention bug described in §3.9.3 — flags any class whose mean relative bounding-box area is less than 35% of the median across classes, as an automatic check for inconsistent annotation conventions (e.g. a blemish-only polygon mixed with whole-tomato bounding boxes).

Background handling. No explicit background-subtraction or segmentation preprocessing step is implemented in the codebase; background variation is instead addressed through the controlled lightbox capture protocol (§3.5) and, for the live-inference path, a secondary object-detection filter (§3.15) that suppresses non-tomato objects in-frame rather than modifying the background itself.

INSERT FIGURE 3.9: IMAGE PREPROCESSING AND AUGMENTATION PIPELINE

Figure 3.9. Preprocessing pipeline: raw image → resize to 640×640 → on-the-fly augmentation (training only) → model input. A separate diagram should show the offline defect-class augmentation branch.

INSERT FIGURE 3.10: BEFORE/AFTER EXAMPLE OF AUGMENTED DEFECT-CLASS IMAGES

Figure 3.10. Example defect-class training image before and after offline augmentation (flip/rotate/scale/brightness/hue jitter).

3.9 Image Annotation

3.9.1 Annotation Tool and Format

All images were annotated using **Roboflow**, a web-based annotation and dataset-versioning platform. Annotations were exported in YOLOv8 format (one `.txt` label file per image; each line specifies a class index and a normalised bounding box: class, x-center, y-center, width, height).

3.9.2 Annotation Convention

Each tomato instance in an image is annotated individually with its own whole-tomato bounding box and a single class label drawn from the five target classes (§3.7); an image containing multiple tomatoes therefore contains multiple separate annotations. For the defect class specifically, the convention is a whole-tomato bounding box (not a polygon isolating only the blemished region) labelled `defect` regardless of the fruit's underlying ripeness colour — ripeness-versus-defect ambiguity is resolved through the class label together with the learned visual content inside a consistently sized box, not through box geometry.

3.9.3 Annotation Quality Control and Correction

Two distinct data-quality issues were identified during model evaluation and corrected through re-annotation, and are documented here because they materially shaped the final annotation and dataset-preparation procedure:

Issue 1 — mixed annotation geometry (identified and fixed). In an earlier dataset version, some defect-class images (sourced from a public dataset) were annotated with a polygon around only the blemished region, while the project's own images used whole-tomato bounding boxes as described above. This inconsistency broke the object detector's implicit assumption of consistent per-class object-scale statistics, and was the primary cause of the defect class's initially weak performance (test-set mAP50 of 0.595). The correction — re-annotating all defect images with whole-tomato bounding boxes on the same convention as the other four classes — raised defect mAP50 to 0.940 on that dataset version.

Issue 2 — domain/lighting mismatch (identified and mitigated). Even after Issue 1 was corrected, live-camera testing revealed that healthy breaker/turning/red tomatoes were being misclassified as defect. Investigation traced this to the fact that the defect class's images were still, at that point, largely sourced from a public dataset (varied backgrounds/lighting), while the other four classes were captured entirely under the project's own controlled lightbox conditions — the model had partly learned "different lighting/background domain" as a spurious cue for "defect," rather than purely learning blemish appearance. This was corrected by removing the mismatched public-dataset defect images, capturing new self-photographed, whole-box-annotated defect images under the same lightbox conditions as the other classes, and applying the class-balancing augmentation described in §3.8 to compensate for the smaller resulting defect image count. This is the dataset export (`tomato_project.v6-version_05.yolov8`, "v6_balanced") used to train the currently deployed model, and its evaluation is reported in §3.14.

This history is included because it reflects a genuine, evidence-driven quality-control cycle in the annotation process, and because a residual limitation it revealed (ripeness-class boundary confusion between breaker/turning/green — §3.25) remains an open finding of the study.

INSERT FIGURE 3.11: EXAMPLE OF AN ANNOTATED TOMATO IMAGE IN ROBOFLOW

Figure 3.11. Example whole-tomato bounding-box annotation, shown for a multi-tomato image with mixed classes.

Detailed, step-by-step Roboflow annotation procedure (tool navigation, label-drawing conventions, and reviewer sign-off steps) is provided in **Appendix A**.

3.10 Dataset Splitting

The dataset was split by Roboflow at export time into training, validation, and test subsets. The exported base counts are shown in Table 3.7; the training split was subsequently supplemented with the offline-augmented defect images described in §3.8, while the validation and test splits were left unmodified so that reported evaluation metrics remain an honest, unaugmented measure of model performance.

Table 3.7. Dataset split.

Dataset	Number of Images (base export)	Percentage of base export	Notes
Training	5,498	66.9%	+ 668 offline-augmented defect images (train-only supplement)
Validation	1,574	19.2%	Used for early stopping and hyperparameter/model-selection decisions during training
Testing	777	9.5%	Held out completely from training; used only for final reported performance (§3.14)
Total	7,849	100%*	<i>Percentages calculated directly from the exported image counts.</i>

Deviation from the originally proposed split ratio. The project's formal research proposal (Tomato_Progress.pdf, §4.3 "Dataset Split") specified a target split of **70% training / 10% validation / 30% test**, deliberately using a larger-than-usual test set "to provide a sufficiently large training set while keeping a relatively large held-out test set for unbiased performance evaluation." The split actually produced by the Roboflow export used for the deployed model (66.9% / 19.2% / 9.5%, above) differs substantially from this target, particularly in allocating far less to the test split (9.5% vs. the planned 30%) and more to validation (19.2% vs. the planned 10%). **[INFORMATION REQUIRED]**: if this deviation was a deliberate later decision (e.g. favouring a larger validation set for early-stopping stability over a very large test set), state the reasoning here; otherwise report it as an unexplained deviation from the proposed methodology, since the "do not assume a procedure was followed as planned" instruction applies equally to assuming a deviation was intentional.

Dataset splitting into independent training, validation, and test subsets is standard practice in supervised deep learning: the training set is used to fit model parameters, the validation set is used during training to monitor generalisation and to select the best checkpoint (via early stopping), and the test set — never

seen during training or checkpoint selection — is used exactly once to report an unbiased estimate of real-world performance. Keeping the test set fully held out (including from the offline defect-class augmentation) was a deliberate methodological choice to avoid data leakage inflating the reported results.

3.11 AI Model Development

The system uses **YOLOv8** (You Only Look Once, version 8), a single-stage convolutional object-detection architecture, for simultaneous tomato localisation and classification. Specifically, the **YOLOv8m ("medium") variant** (`yolov8m.pt` base weights) was used for the currently deployed model.

YOLOv8 was selected because it performs detection and classification in a single forward pass, directly predicting bounding boxes and class probabilities for every object in a frame without a separate region-proposal stage. This is well suited to the project's requirement for real-time, full-frame, multi-instance detection: a single camera frame may contain more than one tomato at different ripeness stages simultaneously, and the sorting system needs a bounding box, a class label, and a confidence score for each one, at a frame rate compatible with conveyor-belt speeds.

At inference time, the model takes a single RGB video frame as input; each detected tomato is returned as an axis-aligned bounding box, a predicted class (one of the five classes in §3.7), and a confidence score. Multiple tomatoes in the same frame are each assigned an independent detection. The model does not perform any additional post-hoc classification step — the ripeness/defect decision is the YOLOv8 class prediction itself, subsequently aggregated across frames by the tracking logic described in §3.15.

Two earlier training runs (`runs_local/tomato_5class_local/`, and the original Kaggle-trained baseline in `kaggle/TOMATO_MODEL_RESULTS/`) are retained in the repository for internal comparison but are **not** the deployed model and are not the basis of the results reported in §3.14. An alternative YOLO26m-based retrain also exists (`runs_local/tomato_5class_yolo26m_v6_balanced/`) but likewise is not the deployed configuration; **[INFORMATION REQUIRED: if a YOLOv8-versus-**

YOLO26 comparison is to be reported as part of this thesis's results, its evaluation methodology and numbers should be added as a separate subsection or appendix, sourced from `YOLO26M_VS_YOLOV8M_COMPARISON.md` and the corresponding training logs.]

INSERT FIGURE 3.12: YOLOV8 DETECTION WORKFLOW/ARCHITECTURE DIAGRAM

Figure 3.12. Simplified YOLOv8 single-stage detection workflow: input frame → backbone/neck feature extraction → detection head → bounding boxes with class and confidence.

3.12 Model Training

3.12.1 Training Environment

Training was performed **locally**, not on a cloud service, on a workstation equipped with an **NVIDIA GeForce RTX 3070 Ti GPU (8,192 MiB VRAM)**, running Python 3.11.9, PyTorch 2.11.0 (CUDA 12.8 build), and Ultralytics 8.4.104 (confirmed in `train_logs/v6_balanced_train.log`: `Ultralytics 8.4.104 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 3070 Ti, 8192MiB)`). This is a deliberate departure from the project's earlier Kaggle-hosted training (dual-GPU cloud notebook); `train_local.py` was written specifically to adapt the training configuration to a single 8 GB local GPU, including an automatic-batch-size setting and a pre-training VRAM check.

3.12.2 Model Configuration and Hyperparameters

Table 3.8. Training configuration for the deployed model (`tomato_5class_v6_balanced`), taken directly from `args.yaml` .

Parameter	Value
Base model	YOLOv8m (<code>yolov8m.pt</code>)
Training data	<code>data_balanced.yaml</code> (5,498 base + 668 augmented defect images)
Epochs (maximum)	150
Early-stopping patience	25 epochs
Epochs actually run	77 (early-stopped; best checkpoint at epoch 52)
Training duration	4.568 hours
Batch size	Auto (<code>batch=-1</code> , Ultralytics auto-batch selection for available VRAM)
Image size	640 × 640
Optimizer	Auto (Ultralytics automatic optimizer selection)
Initial learning rate (lr0)	0.01
Final learning rate factor (lrf)	0.01
Momentum	0.937
Weight decay	0.0005
Warm-up epochs	3.0
Loss weights	box = 7.5, cls = 0.5, dfl = 1.5
Augmentation	See §3.8
Model size (fused, post-training)	93 layers, 25,842,655 parameters, 78.7 GFLOPs

The hyperparameters were carried over from a previously validated Kaggle training configuration for this dataset (which had achieved 87.1% mAP50), adjusted only where necessary for single-GPU, 8 GB-VRAM hardware (`train_local.py` inline documentation).

3.12.3 Training Procedure

Training was executed via `train_local.py`, which performs the following steps in order: (1) a pre-flight check confirming PyTorch/Ultralytics installation, CUDA availability, and at least 3.0 GB of free VRAM; (2) the dataset audit described in §3.8; (3) `model.train()` with the configuration in Table 3.8, monitored against the validation split each epoch, with the best-performing checkpoint (by validation metric) saved as `best.pt` and intermediate checkpoints saved every 5 epochs; (4) automatic early stopping once 25 consecutive epochs passed without validation improvement (triggered after epoch 77, best epoch 52); (5) a final, separate evaluation pass of `best.pt` against the **held-out test split** (§3.14), reported independently from the validation metrics used for checkpoint selection.

INSERT TRAINING SCREENSHOT(S)

Figure 3.13. Training console output showing epoch progress, loss values, and validation metrics.

INSERT FIGURE 3.14: TRAINING LOSS AND METRIC CURVES

Figure 3.14. Training/validation loss curves (box, classification, DFL) and detection metrics (precision, recall, mAP50, mAP50-95) across training epochs. Source file:

`runs_local/tomato_5class_v6_balanced/results.png`.

INSERT FIGURE 3.15: PRECISION-RECALL CURVE

Figure 3.15. Precision-recall curve for the final model. Source file:

`runs_local/tomato_5class_v6_balanced/BoxPR_curve.png`.

3.13 Model Validation

The validation split (1,574 images, 1,596 annotated instances; Table 3.5) was used throughout training to monitor the model's generalisation performance after every epoch, independent of the training data itself. Ultralytics' early-stopping mechanism used the validation metric trend to detect when further training was no longer improving generalisation, halting training automatically once 25 consecutive epochs passed without improvement, and restoring the best-performing checkpoint (epoch 52) as the final model — this is the mechanism used to guard against overfitting in this study, rather than a manually inspected learning-curve cutoff.

Validation-set performance at the selected checkpoint (epoch 52, `best.pt`):

Metric	Overall
Precision	0.925
Recall	0.931

Metric	Overall
mAP@50	0.960
mAP@50–95	0.730

Per-class validation performance: breaker P 0.920 / R 0.923 / mAP50 0.959; defect P 0.853 / R 0.927 / mAP50 0.931; green P 0.978 / R 0.991 / mAP50 0.982; red P 0.980 / R 0.942 / mAP50 0.986; turning P 0.891 / R 0.875 / mAP50 0.943 (`train_logs/v6_balanced_train.log`).

The validation results directly informed one model-selection decision during the project: an earlier training run's evaluation (on validation and test data) surfaced the two defect-class data-quality problems described in §3.9.3, which led to dataset correction and retraining rather than simply accepting a weaker model. The confusion matrix generated from validation/test predictions (§3.14) was the specific diagnostic that revealed the second issue (domain mismatch) was not visible from the aggregate precision/recall/mAP figures alone.

3.14 Model Testing and Evaluation

The test split (777 images, 787 annotated instances) was held out completely from both training and validation-based checkpoint selection, and evaluated exactly once with the selected `best.pt` checkpoint via `model.val(split="test")`, reported here as the official, unbiased performance estimate for the deployed model.

3.14.1 Evaluation Metrics

The following standard object-detection metrics were computed:

- **Precision** = $TP / (TP + FP)$ — the proportion of predicted tomato detections that were correct, for a given class.
- **Recall** = $TP / (TP + FN)$ — the proportion of actual tomato instances of a given class that the model successfully detected.
- **mAP@50** — mean Average Precision at an Intersection-over-Union (IoU) threshold of 0.50, averaged across all five classes; a detection is counted as a true positive if its predicted box overlaps the ground-truth box by at least 50% IoU and the class matches.
- **mAP@50–95** — mean Average Precision averaged across IoU thresholds from 0.50 to 0.95 (in steps of 0.05), a stricter metric that additionally rewards precise bounding-box localisation, not just correct class and rough overlap.
- **F1-score** = $2 \times (Precision \times Recall) / (Precision + Recall)$ — the harmonic mean of precision and recall; used qualitatively (via the F1 curve) to select the deployment confidence threshold (§3.16), rather than reported as a separate headline number here.

3.14.2 Test-Set Results

Table 3.9. Held-out test-set performance, `tomato_5class_v6_balanced` (777 images, 787 instances).

Class	Images	Instances	Precision	Recall	mAP@50	mAP@50–95
Breaker	191	191	0.924	0.937	0.966	0.762
Defect	45	66	0.766	0.818	0.804	0.599
Green	191	191	0.976	0.990	0.982	0.722
Red	190	190	0.987	0.958	0.988	0.724
Turning	149	149	0.880	0.906	0.941	0.749
Overall (mean)	777	787	0.906	0.922	0.936	0.711

Inference speed on the test set: 0.7 ms preprocessing, 6.7 ms inference, 0.7 ms postprocessing per image (measured on the RTX 3070 Ti training/evaluation host).

Compared against the pre-correction defect-class baseline (test mAP50 0.595, produced by the mixed-annotation problem described in §3.9.3), the defect class improved by +0.209 (to 0.804) after the annotation and domain-mismatch corrections. The confusion matrix (Figure 3.16) shows that the remaining, largest source of confusion in the corrected model is between the three adjacent ripeness classes — breaker, turning, and green — which sit on a continuous colour gradient rather than a hard visual boundary, rather than the defect class itself; this is discussed further as a limitation in §3.25.

INSERT FIGURE 3.16: CONFUSION MATRIX

Figure 3.16. Confusion matrix for the test-set evaluation. Source file:
`runs_local/tomato_5class_v6_balanced/test_eval/confusion_matrix.png` (raw counts) and
`confusion_matrix_normalized.png` (row-normalised).

INSERT FIGURE 3.17: MODEL PERFORMANCE GRAPHS

Figure 3.17. Precision, recall, and mAP curves as a function of confidence threshold. Source files:
`BoxP_curve.png` , `BoxR_curve.png` , `BoxF1_curve.png` , `BoxPR_curve.png` .

Full per-epoch metric logs, all early-checkpoint weights, and the complete training console log are provided in **Appendix C**; the F1-curve-based derivation of the deployment confidence threshold (0.45) is provided in **Appendix D**.

3.15 Tomato Detection Process

The end-to-end detection process applied at inference time (as implemented in `conveyor_core.py` and `conveyor_integration.py`) proceeds as follows:

1. **Image/video input** — a live frame is captured from the USB webcam (960×720, 30 FPS).
2. **Preprocessing** — the frame is resized to the model's 640×640 input size (handled internally by the Ultralytics inference call).
3. **YOLOv8 inference** — the frame is passed through the trained model (`model(frame, conf=0.45, imgsz=640, device=...)`), where 0.45 is the deployment confidence threshold, selected as the approximate F1-optimal operating point identified from the training-curve analysis (§3.14).
4. **Tomato detection** — the model returns zero or more detections, each with a bounding box, class, and confidence score.
5. **Non-tomato object filtering** — a secondary, general-purpose object-detection model (`yolo26n.pt` , a COCO-pretrained model, confidence threshold 0.35, IoU threshold 0.3) is run on the same frame to identify common distractor objects (e.g. bottles, cups, hands/other body parts via "person," phones); any tomato-model detection whose box substantially overlaps a detected distractor object is suppressed, reducing false detections of non-tomato objects on or near the belt.
6. **Bounding-box generation** — surviving detections are drawn on the frame for visual feedback (dashboard/OpenCV window) and passed to the tracking logic.
7. **Per-tomato tracking and vote aggregation** — a `TomatoSession` state machine (IDLE/TRACKING) follows a single tomato across consecutive frames (the system assumes single-file tomato flow — one tomato in view at a time), accumulating a class "vote" per frame while the tomato remains in view (tracking ends after `EXIT_GRACE_S = 0.6` seconds without detection, with a minimum of 2 frames required to count a track).
8. **Ripeness/defect classification (finalisation)** — once a track ends, the final class is determined by a confidence-weighted vote across the tomato's tracked frames, with an explicit override rule for the defect class: defect is only selected as final if at least 2 frames voted defect **and** those frames represent at least 35% of the tomato's total tracked frames, each such frame having a defect confidence of at least 0.80 — a deliberately conservative threshold introduced specifically to prevent the domain-mismatch effect described in §3.9.3 from over-triggering false defect calls in live operation.
9. **Output** — the finalised class, its confidence, and (via the static shelf-life lookup, §3.16.4) an estimated shelf-life value are logged to the SQLite database and, if hardware is connected, converted into a sorting command (§3.18).

INSERT FIGURE 3.18: TOMATO DETECTION AND CLASSIFICATION FLOWCHART

Figure 3.18. Camera → frame preprocessing → YOLOv8 inference → non-tomato filtering → per-track vote aggregation → finalised ripeness/defect classification → output (dashboard display + sorting command), matching the implementation in `conveyor_core.py` and `dashboard_app.py` .

3.16 Hardware Prototype Development

The physical prototype is a belt conveyor with a fixed overhead camera and four servo-actuated sorting gates, built incrementally between January and July 2026 according to the monthly progress reports.

3.16.1 Mechanical Structure and Conveyor Mechanism

The conveyor frame was constructed from 1-inch metal box-section bar, following dimensions validated earlier in a SolidWorks CAD model (January–April 2026), and finished with paint for durability and presentation (May–June 2026 Progress Reports). The belt is driven by a NEMA 23 stepper motor, mechanically coupled to the conveyor's drive roller via an 8 mm-bore/20-tooth pulley on the motor shaft, a 10 mm-bore/60-tooth pulley on the roller, and a closed-loop timing belt, with the motor secured by an L-shaped mounting bracket. Six custom bracket-and-bearing assemblies support smooth roller rotation. The conveyor's rolling mechanism was assembled and mechanically tested (alignment and stability issues identified and corrected), and — per the July 2026 Progress Report — was successfully operated under stepper-motor control and functionally tested by transporting fresh tomatoes across the belt with "smooth and consistent" movement.

3.16.2 Camera Mounting

An adjustable, single-axis camera mounting mechanism was custom-designed and fabricated (May–June 2026 Progress Reports) to allow the camera's position and viewing angle to be tuned for field-of-view coverage of the belt.

3.16.3 Sensors and Sorting Mechanism

Four IR (infrared) sensors and four MG995 servo-actuated sorting gates are installed, one pair per target class (green, breaker, turning, red); the defect class is intentionally not assigned a gate — a tomato classified as defect receives no actuation command and simply continues to the end of the belt. This design (documented in the project's engineering notes, `PROJECT.md`) reflects a deliberate single-reject-stream sorting design rather than a limitation.

Confirmed gate/servo wiring (from the `tomato_v2` firmware, `src/main.cpp`, cross-checked against `dashboard_app.py`'s class-to-command mapping): servo 1 (GPIO 13) = green, servo 2 (GPIO 12) = breaker, servo 3 (GPIO 14) = turning, servo 4 (GPIO 27) = red; IR sensors on GPIO 18, 19, 21, 23.

[INFORMATION REQUIRED / verify before submission] — a discrepancy exists between two internal sources regarding the gates' **physical order along the belt**: the project's design notes (dated 2026-07-23) describe the intended order as green → breaker → turning → red, with green closest to the camera; however, an inline comment in the `tomato_v2` firmware's queue-sequencing code (`Machine.h`, dated 2026-08-15, i.e. more recent and stated to be "confirmed on the physical rig") describes the belt order as gate 4 first, then gate 3, gate 2, gate 1 last — which, if gate numbering follows the servo numbering

above (servo 4 = red), would place **red** closest to the camera and **green** last, the reverse of the original design intent. Before finalising this subsection, physically confirm which order is actually built and correct whichever source is out of date.

3.16.4 Controller and Power

The system controller is a single ESP32 development board, running the `tomato_v2` PlatformIO firmware, powered from a 24 V/5 A supply (stepper driver side) and a 12 V/2 A adapter (logic side), per Table 3.2.

3.16.5 Completion Status

Consistent with the requirement to distinguish completed from planned work: as of the July 2026 Progress Report, the **mechanical conveyor drive was complete and functionally tested with real tomatoes**, but full vision-guided sorting (camera detection triggering an actual physical gate diversion of a moving tomato, measured for sorting accuracy) had **not yet been reported as completed** in the progress-report record available for this draft — the July report's "Activities Planned for the Next Period" explicitly lists installing/configuring the servo gates, integrating the trained model with the hardware, and "performing integrated hardware and software testing using real tomato samples" as forward-looking, not-yet-done work at that time.

[INFORMATION REQUIRED]: If full closed-loop testing (camera → model → gate actuation, measured against real tomatoes) has since been completed, provide the trial date(s), number of tomatoes tested, and sorting-accuracy results, so that §3.21 (System Testing) can report it as completed work rather than as planned work.

INSERT FIGURE 3.19: PROTOTYPE DESIGN (CAD)

Figure 3.19. Conveyor prototype CAD design. Source: `prototype design/prototype desing` exported from `cad.onshape.com/` .

INSERT FIGURE 3.20: ASSEMBLED HARDWARE PROTOTYPE

Figure 3.20. Assembled conveyor prototype — frame, belt drive, camera mount, and sorting gates. Source: `prototype design/` (WhatsApp photographs/video, 2026-07-23) and progress-report figures (motor/pulley/belt assembly, July 2026).

INSERT FIGURE 3.21: CONVEYOR DRIVE SYSTEM DETAIL

Figure 3.21. NEMA 23 stepper motor, pulley, and timing-belt drive assembly.

Detailed hardware specifications (full component datasheets, exact fastener sizes, and wiring diagrams) are provided in **Appendix F**.

3.17 Software System Development

The software system is a Python application built around a Streamlit dashboard (`dashboard_app.py`), backed by the detection/classification logic in `conveyor_core.py` and a SQLite database (`database.py`).

User interface. The dashboard (`st.set_page_config(page_title="Conveyor Sorting Dashboard", layout="wide")`) displays: a live camera feed with drawn bounding boxes; a "current session" panel showing the tracker state (idle/tracking), the running count of tomatoes processed in the session, and the most recently classified tomato's class and confidence; a KPI panel (selectable time window: last 10, 60, or 360 minutes, or all logged history) showing four summary metrics — total tomatoes processed, throughput (tomatoes/minute), defect ratio (percentage and count), and average estimated shelf life (days); a Plotly pie chart of class distribution; and a table of the most recent detections (ID, class, estimated shelf life, confidence, timestamp).

Model loading. The YOLO model is loaded once and cached (`st.cache_resource`) for the session; a dropdown lets the operator select among any trained checkpoint found under `runs_local/*/weights/best.pt` plus the original Kaggle baseline, defaulting to the model path configured in `conveyor_core.py` (`tomato_5class_v6_balanced/weights/best.pt`).

Prediction and result display. Each frame is run through the loaded model with an operator-adjustable confidence-threshold slider (range 0.05–0.95, default 0.45); detected boxes are colour-coded by class (green `#00FF00` , breaker `#FFD700` , turning `#FFA500` , red `#FF0000` , defect `#808080`) and overlaid on the live feed.

Data processing and logging. Every finalised tomato classification (§3.15) is written to the SQLite database (`tomato_detections.db`) together with its confidence, bounding box, timestamp, and an estimated shelf-life value looked up from a static per-class table (§3.16 below); the dashboard's KPI panel reads from this same database to compute aggregate statistics.

Communication with hardware. The dashboard supports sending sorting commands to the ESP32 over a serial/Bluetooth connection, described in §3.18.

INSERT FIGURE 3.22: SOFTWARE ARCHITECTURE DIAGRAM

Figure 3.22. Software architecture: camera input → YOLOv8 inference (`conveyor_core.py`) → per-tomato classification → SQLite logging (`database.py`) → Streamlit dashboard display (`dashboard_app.py`) → serial/Bluetooth command to ESP32.

INSERT FIGURE 3.23: DASHBOARD USER INTERFACE

Figure 3.23. Streamlit dashboard, showing the live feed, session panel, and KPI metrics. Source: [live dashboard results-2026.07.29/](#) (post-correction validation screenshots, taken inside the lightbox on 2026-07-29).

3.18 Hardware–Software Integration

Two distinct communication paths exist in the codebase, reflecting two stages of the project's integration work; both are described here for completeness, with the currently active one identified explicitly.

Currently active path — Bluetooth serial link to `tomato_v2` firmware. The dashboard's `DirectClassSerialSender` opens a serial connection (Bluetooth SPP, device name `TomatoSorter`, 115200 baud) to the ESP32 running the `tomato_v2` firmware, and sends plain-text commands — `class1` (green), `class2` (breaker), `class3` (turning), `class4` (red), or `noclass` (defect) — each with a sequence number and an acknowledgement/retry protocol (up to 4 attempts, 0.5 s timeout per attempt) to tolerate Bluetooth packet loss. On the firmware side, `tomato_v2`'s `Machine` module queues incoming class commands per gate and fires each gate servo ($0^\circ \rightarrow 120^\circ$, per `Settings.h`, held open for 2,000 ms) once the corresponding tomato reaches that gate's IR sensor, with the queue design explicitly accounting for multiple tomatoes in transit at once.

Earlier/simulated path — G/B/T/R timed protocol. `conveyor_core.py`'s `SerialSender / EventScheduler` and `conveyor_integration.py` implement an alternative, simpler protocol (single-character commands G/B/T/R sent to a simpler JSON-based firmware in `esp32_firmware/tomato-project/`), where the gate-fire time is computed in advance from an assumed constant belt speed and fixed gate distances, rather than being triggered by a physical IR sensor. This path defaults to a fully simulated serial connection (`force_simulated=True` in the dashboard; a `--live` flag in `conveyor_integration.py` to enable a real port) and its belt-speed and gate-distance constants (`BELT_SPEED_CMS = 10.0` cm/s, `CAMERA_TO_FIRST_GATE_CM = 20.0` cm, `GATE_SPACING_CM = 15.0` cm, in `microcontroller_config.py`) are explicitly documented in code comments as **unmeasured placeholders**, and the code prints a runtime warning if they are left at these default values.

Per your confirmation, the `tomato_v2` /IR-sensor-triggered path is the one actually wired into the physical prototype at the time of writing; the G/B/T/R timed-scheduling path should therefore be described as an earlier implementation/design exercise (useful for documenting the FIFO event-scheduling logic that was later superseded by the IR-sensor-triggered queue in `tomato_v2`), not as the current production integration.

Implemented, tested, and planned — summary:

- **Implemented and code-complete:** both communication protocols; the `tomato_v2` firmware's IR-triggered gate queue; the dashboard's live inference, KPI logging, and serial command dispatch.
- **Tested (per available evidence):** YOLOv8 live classification inside the lightbox, validated against ground truth on 2026-07-26 and 2026-07-29

(screenshots retained in `live dashboard results-2026.07.26/` and `live dashboard results-2026.07.29/`); conveyor belt mechanical operation with real tomatoes (July 2026 Progress Report). - **Not yet confirmed as tested (per available evidence):** full closed-loop operation — a live camera classification actually triggering the corresponding physical gate servo to divert a moving tomato on the running conveyor, with measured sorting accuracy. **[INFORMATION REQUIRED — see §3.16.5.]**

INSERT FIGURE 3.24: HARDWARE–SOFTWARE INTEGRATION / COMMUNICATION FLOW

Figure 3.24. Communication flow: dashboard classification → serial/Bluetooth command → ESP32 (tomato_v2) → IR-sensor-triggered gate queue → servo actuation.

3.19 Complete End-to-End System Process

Step 1 — Tomato Sample Preparation. Tomatoes are sourced from the collecting center and visually pre-sorted by ripeness stage/defect condition before photography (§3.4).

Step 2 — Image Acquisition. Each tomato is photographed under controlled LED-lightbox lighting against a white background (§3.5).

Step 3 — Image Preprocessing. Images are resized to the model's input resolution and, at training time, augmented on-the-fly; the defect class additionally receives offline, class-targeted augmentation (§3.8).

Step 4 — Tomato Annotation. Each tomato instance is labelled in Roboflow with a whole-object bounding box and one of the five class labels (§3.9).

Step 5 — Dataset Preparation. The annotated dataset is exported from Roboflow and split into training, validation, and test subsets (§3.10).

Step 6 — AI Model Training. A YOLOv8m model is trained locally on an RTX 3070 Ti GPU using the configuration in Table 3.8 (§3.12).

Step 7 — Model Validation. Validation-split performance is monitored every epoch; early stopping selects the best-generalising checkpoint (§3.13).

Step 8 — Model Testing. The selected checkpoint is evaluated once, exactly, on the held-out test split, producing the metrics in Table 3.9 (§3.14).

Step 9 — Tomato Detection. In deployment, the live camera feed is passed frame-by-frame through the trained model, with a secondary filter suppressing non-tomato detections (§3.15).

Step 10 — Ripeness Classification. Each tomato is tracked across consecutive frames as it passes under the camera; its final class is decided by a confidence-weighted vote across those frames, with a conservative override rule specifically for the defect class (§3.15).

Step 11 — Sorting Decision. The finalised class is mapped to a sorting command (green/breaker/turning/red → the corresponding gate; defect → no gate command) (§3.18).

Step 12 — Mechanical Sorting. The command is transmitted to the ESP32 (`tomato_v2` firmware) over Bluetooth serial; the corresponding gate servo actuates when the tomato reaches that gate's IR sensor, diverting it off the belt (§3.16, §3.18).

Step 13 — System Output. The classification result, confidence score, and estimated shelf life are logged to the SQLite database and displayed on the Streamlit dashboard (§3.17).

Step 14 — System Performance Evaluation. The complete system is evaluated at the model level (Table 3.9) and, where testing has been completed, at the functional/integration level (§3.21).

```
Tomato --> Camera --> Image Frame --> Preprocessing --> YOLOv8 Detection
--> Ripeness/Defect Classification (per-track vote) --> Sorting Decision
--> Gate Servo Actuation --> Sorted Tomato --> Logged Result (dashboard + database)
```

INSERT FIGURE 3.25: COMPLETE END-TO-END SYSTEM WORKFLOW

Figure 3.25. Full system workflow from tomato input to sorted output and logged result.

3.20 Experimental Procedure

Experiment 1 — Model training and held-out test evaluation. - Objective: quantify detection/classification performance of the trained YOLOv8m model on unseen data. - Materials: `tomato_project.v6-version_05.yolov8` dataset (§3.6), YOLOv8m architecture, RTX 3070 Ti training host. - Procedure: as described in §3.12–§3.14. - Independent variable: trained model checkpoint (`best.pt` at epoch 52). - Dependent variables: precision, recall, mAP@50, mAP@50-95 (per class and overall). - Controlled variables: fixed test split (777 images, never used in training/validation-based selection), fixed confidence/IoU evaluation settings (Ultralytics defaults for `model.val()`). - Trials: one held-out evaluation pass (by design — the test set is evaluated once to avoid selection bias). - Result: Table 3.9.

Experiment 2 — Live/lightbox classification validation (post-correction). - Objective: verify, outside the offline test split, that the domain-mismatch correction (§3.9.3) resolved the false-defect misclassification problem observed in an earlier live test. - Materials: `tomato_5class_v6_balanced` model, lightbox setup, live USB webcam feed, dashboard application. - Procedure: tomatoes of known class were presented to the live camera inside the lightbox and classified by the dashboard; results were recorded as screenshots. - Independent variable: tomato presented (known ground-truth class). - Dependent variable: dashboard-reported classification and confidence. - Trials: conducted on 2026-07-26 (pre-correction, revealed the domain-mismatch problem) and 2026-07-29 (post-correction, "Correct Results" recorded). - Result: qualitative confirmation of corrected behaviour, evidenced by screenshots

in `live dashboard results-2026.07.26/` and `live dashboard results-2026.07.29/`.

[INFORMATION REQUIRED: if a quantitative accuracy/sample count was recorded for these sessions (e.g. "N of M tomatoes correctly classified"), provide it so this can be reported as a number rather than qualitatively.]

Experiment 3 — Conveyor mechanical functional test. - Objective: confirm the belt drive mechanism transports tomatoes smoothly and consistently under stepper-motor control. - Materials: assembled conveyor frame, NEMA 23 motor, ESP32 + DM542/DM543 driver, fresh tomatoes. - Procedure: the conveyor was operated under programmed stepper control while fresh tomatoes were placed on the belt and observed in transit. - Dependent variable: qualitative transport stability/consistency (no vision or sorting logic active in this test). - Result: "smooth and consistent" transport reported (July 2026 Progress Report). **[INFORMATION REQUIRED: exact number of trial tomatoes/runs, and any quantitative timing measurement (e.g. belt speed in cm/s), which is also needed to replace the placeholder belt-speed constant noted in §3.18.]**

Experiment 4 — Shelf-life observation study. - Objective: establish an expected remaining shelf-life duration for tomatoes at each ripeness/defect class, at the point of sorting. - Owner: K.R.D.H. Gunawardhana (UWU/IIT/21/017), who leads shelf-life prediction and KPI monitoring for the project. - Procedure (confirmed, `Tomato_Progress.pdf` §5.5 "Shelf-Life Estimation"): Padma tomatoes at known ripeness stage were observed under **room-temperature conditions** over a **31-day time-series period**, with **daily morning observations**. Colour differentiation was used at each observation to track the tomato's progression through the four ripeness stages (Green → Breaker → Turning Red → Red), with a tomato considered to have left the study once it became unsellable (excessive softening, fungal/rotting development, or severe blemishes — `Tomato_Progress.pdf` §4.5). - Result — stage durations: the time spent in each ripening stage was measured as **Green = 2 days, Breaker = 5 days, Turning Red = 12 days, Red = 12 days** (summing to the full 31-day observation period). - Result — remaining shelf life: the **remaining** shelf life reported by the system for a tomato detected at a given stage is the sum of that stage's own remaining duration plus all subsequent stages' durations, i.e. cumulative time-to-spoilage from that point onward. This arithmetic exactly reproduces the values hardcoded in `database.py`'s `SHELF_LIFE` table: - Detected at **Green**: $2 + 5 + 12 + 12 = 31$ days remaining. - Detected at **Breaker**: $5 + 12 + 12 = 29$ days remaining. - Detected at **Turning**: $12 + 12 = 24$ days remaining. - Detected at **Red**: $12 = 12$ days remaining. - Detected as **Defect**: **0 days** (rejected at sorting; not part of the observation study, treated as immediately non-marketable by system design). - **[INFORMATION REQUIRED]**: the number of tomato samples observed per ripeness stage, and the storage relative-humidity conditions (only "room temperature" is specified in the source), were not stated in `Tomato_Progress.pdf` or elsewhere in the material reviewed for this draft. If available from K.R.D.H. Gunawardhana's own records, they should be added here to complete the protocol description.

Full raw experimental data (per-trial logs, all screenshots, and the complete shelf-life observation dataset) should be placed in **Appendix B**.

3.21 System Testing

The project's formal research proposal (Tomato_Progress.pdf , §4.8 "Testing and Evaluation") planned system testing in four phases: (1) controlled-environment tests under white-box/LED lighting to validate vision-model performance and tune the capture setup, (2) stress tests varying tomato orientation, occlusion, capture distance, and lighting (without conveyor operation), (3) field validation in a realistic post-harvest/collecting-center environment (described as a "future phase" in the proposal), and (4) benchmarking AI-based grading against manual grading (also a "future phase"). Against this plan, the test cases in Table 3.10 below correspond to: T-01 and T-02/T-03 to phase 1 (controlled/lightbox testing, completed); T-04 to a mechanical-only functional test not explicitly in the original four-phase plan; and T-05–T-08 (end-to-end integration/field-style testing) to phases 2–4, which the proposal itself always described as later-stage work — consistent with these rows still being open at the time of this draft, not a departure from plan.

Table 3.10. System test cases (status reflects evidence available at time of writing).

Test ID	Test Description	Input	Expected Result	Actual Result	Status
T-01	Held-out test-set detection/classification accuracy	777 unseen labelled test images	High precision/recall/mAP across all 5 classes	mAP50 0.936, mAP50-95 0.711, P 0.906, R 0.922 (Table 3.9)	Passed / Completed
T-02	Live classification inside lightbox, pre-correction	Known-class tomatoes, live camera	Correct class reported	Breaker/turning/red frequently misclassified as defect (domain-mismatch issue)	Completed — Failed, led to correction (§3.9.3)
T-03	Live classification inside lightbox, post-correction	Known-class tomatoes, live camera	Correct class reported	Correct results recorded (2026-07-29)	Passed / Completed (see [INFORMATION REQUIRED] note in Experiment 2 for a quantitative count)
T-04	Conveyor belt mechanical transport	Fresh tomatoes on running belt	Smooth, stable transport	Smooth and consistent transport confirmed	Passed / Completed
T-05	Non-tomato object rejection (distractor filter)	Frame containing tomatoes and common non-tomato objects	Non-tomato objects not classified as a tomato class	[INFORMATION REQUIRED: no recorded trial found]	[INFORMATION REQUIRED]
T-06	Sensor/gate actuation (single gate, bench test, no belt motion)	Direct serial command to one gate class	Corresponding servo opens and auto-returns	[INFORMATION REQUIRED: no recorded trial found]	[INFORMATION REQUIRED]

Test ID	Test Description	Input	Expected Result	Actual Result	Status
T-07	End-to-end sorting accuracy (camera → classification → physical gate diversion, belt running)	Real tomatoes of known class, moving on the belt	Tomato is physically diverted at the correct gate	[INFORMATION REQUIRED — see §3.16.5]	[INFORMATION REQUIRED — not yet confirmed complete]
T-08	Bluetooth command reliability (ACK/retry) under real conditions	Sequenced class1–4/noclass commands over the live Bluetooth link	Commands delivered despite packet loss (via retry protocol)	[INFORMATION REQUIRED: no recorded trial found]	[INFORMATION REQUIRED]

Testing categories represented above: detection-accuracy testing (T-01), classification testing (T-02, T-03), functional/mechanical testing (T-04), and integration/end-to-end testing (T-05–T-08, largely still pending confirmation). If the missing trials in Table 3.10 have since been run, supply their dates, sample sizes, and outcomes so the table can be completed with real results rather than placeholders — this table should not be filled in with assumed "Passed" statuses for rows not yet backed by evidence.

Detailed, per-image test-set predictions and the complete confusion-matrix data underlying T-01 are provided in **Appendix D**.

3.22 Data Analysis

Model performance was analysed using the standard object-detection metrics defined in §3.14.1 (precision, recall, mAP@50, mAP@50-95), computed per class and as a class-averaged mean by the Ultralytics evaluation API, and visualised through the training-curve and confusion-matrix figures referenced throughout this chapter. The confusion matrix specifically was used as a diagnostic (not just a summary) tool during the study — it was the artefact that revealed the ripeness-class boundary confusion (breaker/turning/green) as the model's principal remaining weakness, a distinction not visible from the per-class mAP table alone (§3.14.2).

System-level (KPI) data — total tomatoes processed, class-distribution counts, defect ratio, throughput, and average shelf life — is computed by simple aggregation (counts, means, ratios) over the SQLite detection log (`database.py::get_statistics()`), displayed live on the dashboard rather than analysed through formal inferential statistics.

[INFORMATION REQUIRED]: No hypothesis testing, confidence-interval estimation, or other inferential statistical technique was found implemented in the codebase or reported in the progress reports reviewed for this draft. If your thesis is expected to report statistical significance (e.g. for a claimed improvement between model versions, or for the shelf-life study in §3.20/Experiment 4), clarify which comparisons need formal statistical treatment (e.g. a paired test comparing pre- and post-

correction defect performance) so the appropriate method can be specified here — this should not be added without your confirmation of what comparison is actually needed, per the instruction not to introduce statistical techniques that were not actually used.

3.23 Research Workflow Diagram

```
Sample Collection
|
Image Acquisition
|
Image Preprocessing
|
Annotation
|
Dataset Preparation
|
Model Training
|
Validation
|
Testing
|
Model Evaluation
|
[Correction loop if evaluation reveals a systematic problem -- see S:3.9.3]
|
Prototype Development
|
System Integration
|
Tomato Detection
|
Ripeness Classification
|
Sorting Decision
|
Final Sorting
|
System Evaluation
```

INSERT FIGURE 3.26: RESEARCH WORKFLOW DIAGRAM

Figure 3.26. Full research workflow, reproduced as a formatted diagram from the structure above.

3.24 Ethical and Safety Considerations

This research involved food produce (tomatoes) and electromechanical prototype hardware; it did not involve human or animal subjects, personal data collection, or any procedure requiring institutional ethics-board review. Relevant considerations were therefore engineering-safety and food-handling practices rather than research-ethics approvals:

- **Electrical safety:** the prototype operates multiple power domains (24 V/5 A for the stepper driver, 12 V/2 A logic supply, and the servos' own supply); wiring and connections should be inspected before each powered test, consistent with standard electromechanical prototyping practice.
- **Mechanical safety:** moving parts (belt, roller, stepper motor, actuating gate servos) present pinch-point risk during operation and testing; the belt/roller area should be kept clear of hands during powered tests.
- **Food handling:** tomato samples used for photography and conveyor testing should be handled and disposed of hygienically; no chemical treatment or destructive testing of the fruit itself was part of the described methodology beyond the shelf-life observation study (§3.20, Experiment 4), which by nature involves fruit degrading to spoilage under observation.
- **Data/IP:** the defect-class images originally sourced from a public (Kaggle) dataset (§3.9.3) were used under that dataset's terms; the project's own Roboflow-hosted dataset is licensed CC BY 4.0.

[INFORMATION REQUIRED]: confirm whether your department/faculty requires a formal safety-declaration form for lab-based electromechanical prototyping (common in engineering-adjacent undergraduate theses), and if so, reference or attach it here rather than relying on the general statement above.

3.25 Limitations of the Methodology

The following limitations follow directly from evidence gathered during the study, rather than being generic caveats:

1. **Residual ripeness-class confusion.** Even after correcting the defect-class data-quality issues, the confusion matrix shows breaker, turning, and green are still confused with each other more than any class is confused with defect — an expected consequence of ripeness existing on a continuous colour gradient rather than discrete visual categories, and a genuine, not yet resolved, limitation of a hard multi-class classification framing.
2. **Unmeasured belt-speed/gate-timing constants in the legacy protocol.** The G/B/T/R timed-scheduling integration path (§3.18) uses belt-speed and gate-distance constants that are explicitly documented in code as unmeasured placeholders; this path is not the one currently wired to the physical prototype, but the placeholder values should not be mistaken for calibrated figures if that path is referenced anywhere else in the thesis.

3. **End-to-end sorting-accuracy evidence gap.** At the time of writing, the repository and progress reports available for this draft do not contain a confirmed, completed trial of the full closed loop (live camera detection triggering an actual physical gate diversion, with measured accuracy) — see §3.16.5 and Table 3.10, row T-07. This should either be completed and reported, or explicitly scoped out of the thesis's claimed contributions if it will not be completed before submission.
4. **Single-file-flow assumption.** The tracking and classification logic assumes one tomato is in the camera's field of view at a time; simultaneous multi-tomato tracking/identity management is not implemented, which constrains achievable sorting throughput compared to a system with per-object identity tracking.
5. **Dataset domain scope.** All training images were captured under a single controlled lightbox setup at one collection location; performance under substantially different real-world lighting/backgrounds beyond what was validated in §3.20 (Experiment 2) has not been separately quantified.
6. **Shelf-life estimation is a static lookup, not a predictive model.** The reported shelf-life figures are a fixed per-class table derived from one 31-day observation study, not a per-tomato predictive estimate conditioned on individual fruit condition; this should be stated explicitly if the thesis title's "shelf-life estimation" claim is to be precisely scoped.

3.26 Summary of Figures and Tables

For thesis-formatting convenience, all figures and tables referenced in this chapter are listed here; renumber to match the final document's global figure/table numbering.

Figures: 3.1 Overall methodology flowchart · 3.2 Hardware components/prototype assembly · 3.3 Sample collection process · 3.4 Image acquisition setup · 3.5 Example images per class · 3.6 Instance distribution per class/split · 3.7 Train/valid/test distribution · 3.8 Ripeness-stage visual examples · 3.9 Preprocessing/augmentation pipeline · 3.10 Defect-class augmentation before/after · 3.11 Annotated image example · 3.12 YOLOv8 detection workflow · 3.13 Training console output · 3.14 Training loss/metric curves · 3.15 Precision–recall curve · 3.16 Confusion matrix · 3.17 Model performance curves · 3.18 Detection/classification flowchart · 3.19 Prototype CAD design · 3.20 Assembled hardware prototype · 3.21 Conveyor drive detail · 3.22 Software architecture diagram · 3.23 Dashboard UI · 3.24 Hardware–software integration flow · 3.25 End-to-end system workflow · 3.26 Research workflow diagram.

Tables: 3.1 Materials/samples · 3.2 Hardware components · 3.3 Software/technologies · 3.4 Dataset summary · 3.5 Per-class instance counts by split · 3.6 Ripeness/quality class definitions · 3.7 Dataset split · 3.8 Training configuration · 3.9 Test-set performance · 3.10 System test cases.

3.27 Appendix Guidance

Following the thesis formatting requirement that lengthy standard procedures and detailed data tables be moved out of the main chapter, the following appendices are recommended:

- **Appendix A — Detailed Dataset and Annotation Procedure.** Full Roboflow annotation walkthrough, class-labelling rules, and quality-review steps (summarised in §3.9).
- **Appendix B — Complete Experimental Data.** Raw logs and screenshots for Experiments 2–4 (§3.20), including the full shelf-life observation dataset once obtained.
- **Appendix C — Detailed Model Training Parameters and Logs.** Full `args.yaml`, complete per-epoch `results.csv`, and the full training console log (summarised in §3.12).
- **Appendix D — Extended Test Results.** Full per-image test-set predictions, the complete confusion matrix data, and the F1-curve threshold-selection analysis (summarised in §3.14, §3.15).
- **Appendix E — Additional System Images.** Extended photo/video documentation of the prototype build (from `prototype design/` and the monthly progress reports).
- **Appendix F — Detailed Hardware Specifications.** Full component datasheets, wiring diagrams, and PlatformIO firmware source listings (summarised in §3.16).
- **Appendix G — Standard Analytical Procedures.** Any established, standard procedure referenced but not itself novel (e.g. standard YOLO training/evaluation procedure documentation), if required by your department's format guidelines.